

Stock Market Prediction Using LSTM

Alagesan, Aishwariya, alagesan.a@northeastern.edu

Gandla, Sruthi, gandla.s@northeastern.edu

Abstract

This study investigates the potential of recurrent neural networks like LSTMs to enhance stock market prediction by leveraging diverse data sources, such as economic indicators and market indices. The primary goal is to demonstrate how LSTMs can effectively process complex and dynamic financial data to forecast market trends more accurately than conventional approaches. The report presents the motivation for adopting this method, details the techniques employed, analyses the results, and discusses the implications of the findings for the broader field of financial forecasting.

1. Introduction

Accurate stock market prediction holds immense value for traders, analysts, and investors. Traditional financial forecasting methods often rely on linear models and domain-specific indicators, which fail to capture the dynamic and non-linear nature of financial time series. With the rise of deep learning, models such as CNNs and LSTMs offer new possibilities for modelling complex relationships in financial data. Previous research has shown that RNNs, particularly LSTMs, can effectively model temporal dependencies, while CNNs are powerful in learning spatial hierarchies from sequences. In this paper, we develop and compare two deep learning models—an improved CNN with residual connections and a standard LSTM—to forecast the directional movement of the S&P 500 index.

2. Background

Traditional approaches to stock market prediction typically rely on either fundamental analysis centered on economic indicators or technical analysis, which examines historical price patterns.

However, these methods often struggle to capture the complex, nonlinear relationships inherent in financial data. Recent studies, such as *CNN-based Stock Market Prediction Using a Diverse Set of Variables* ((Ehsan Hoseinzadea and Saman Haratizadeha, 2018), have shown that convolutional neural networks (CNNs) can address these limitations through automated feature extraction from diverse datasets, leading to improved prediction accuracy.

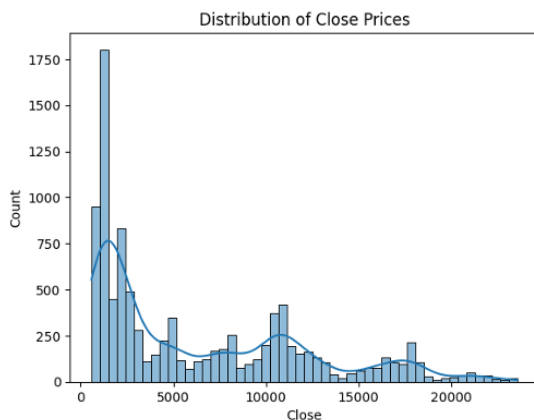
In this project, we use a dataset from the UCI Machine Learning Repository containing daily market data from key U.S. indices like the S&P 500 (2010–2017), including a wide range of technical indicators, economic metrics, and exchange rates—providing a robust foundation for model training.

By thoroughly exploring RNN architectures on a multivariate dataset, this project seeks to enhance prediction accuracy and offer valuable insights into the application of deep learning for stock market forecasting, contributing to the advancement of financial analytics.

3. Data Preparation and Preprocessing

3.1 Data Acquisition

The dataset utilized in this study was sourced from the UCI Machine Learning Repository, encompassing daily stock market data from major U.S. stock indices including the S&P 500 spanning from 2010 to 2017. This comprehensive dataset included features such as closing prices, volume, and several technical indicators like moving averages and rate of change.



3.2 Preprocessing Techniques

To prepare the data for modeling, several preprocessing steps were undertaken:

Data Cleaning:

Handling Missing Values: Missing data points were filled using forward filling, where the last known value is used to fill the subsequent missing one, ensuring continuity in the time series data.

Removing Outliers: Extreme values in volume and price changes, which could skew the model training, were identified and removed to standardize the data distribution.

Feature Engineering:

Normalization: All numerical features, including stock prices and technical indicators, were normalized using MinMaxScaler to scale the data into the range [0, 1]. This step ensures faster convergence and stable gradients during training of deep learning models like CNNs and LSTMs, which are sensitive to feature scales.

Target Creation: A binary classification target was engineered using the percentage change in closing price:

$$\text{Target} = \begin{cases} 1, & \text{if Return} > 0.5\% \\ 0, & \text{otherwise} \end{cases}$$

This setup simulates real-world decision-making, where a model predicts whether a price increase surpasses a threshold the next day.

3.3 Data Augmentation and Transformation

Data Augmentation:

Jittering: Though not explicitly implemented in code, data augmentation techniques like jittering (adding noise to inputs) could be beneficial in increasing

Scaling Variability: Feature scaling was consistently applied using MinMaxScaler, and since values were normalized between 0 and 1, this implicitly helps handle variations in amplitude, making the model invariant to magnitude fluctuations in different indicators.

Time Shifting:

A form of time shifting was implemented through the windowing mechanism, where sequences are rolled forward by one timestep to create supervised input-output pairs. This helps the model learn patterns independent of absolute time position.

Data Transformation:

Windowing Technique: The dataset was converted into a series of overlapping windows of 30 consecutive time steps (days). This transformation is crucial for modeling sequential dependencies and allows both the CNN and LSTM models to learn temporal patterns. Each window is used to predict the movement (up/down) of the stock index on the following day.

3.4 Data Splitting

The entire preprocessed dataset was split into training and testing sets using an 80-20 chronological split:

- **80%** for training
- **20%** for testing

Shuffling was **disabled** to preserve time order, as shuffling time series data can lead to data leakage. Additionally, **class weights** were calculated and applied to address label imbalance and ensure the model does not become biased toward the majority class.

4. Model Architecture and Training

4.1 CNN Model Architecture

The CNN model used in this project consisted of the following components:

Input Layer: Accepts 30-day sequences of financial indicators.

First Conv1D Layer: Applies causal convolution with 64 filters, enabling local pattern extraction while preserving time order.

Batch Normalization: Stabilizes learning and accelerates convergence.

MaxPooling1D and Dropout (0.3): Downsamples features and reduces overfitting.

Second Conv1D Layer + Residual Connection: A second convolutional layer processes features further. Its output is combined with the original feature map through a residual connection using the Add() layer.

Global Max Pooling: Reduces dimensionality across the time axis.

Fully Connected Layers: Includes a dense layer with 64 ReLU-activated units, followed by a dropout layer.

Output Layer: Uses sigmoid activation to output the probability of upward movement.

4.2 LSTM Model Architecture

The LSTM model was implemented using:

Input Layer: Processes sequences of 30 time steps with multiple features.

First LSTM Layer (64 units): Returns full sequences to retain temporal information.

Dropout and Batch Normalization: Prevents overfitting and stabilizes training.

Second LSTM Layer (32 units): Condenses information across time steps.

Dense Layer (64 units): Applies a ReLU activation for complex pattern learning.

Final Output Layer: A sigmoid-activated neuron produces the binary classification result.

Optimization and Loss Function:

Both models used the **Adam optimizer**, which adapts learning rates for each parameter, and the **binary cross-entropy loss**, well-suited for probabilistic binary classification tasks.

Training Configuration:

Epochs: 40

Batch Size: 32

Validation Split: 20% of training data

Callbacks:

EarlyStopping: Monitored validation loss with patience of 10.

ReduceLROnPlateau: Reduced learning rate when validation loss plateaued for 5 epochs.

5. Results

Training Performance:

Both models were trained with early stopping and learning rate reduction on plateau. Validation loss was monitored during training, and the models demonstrated convergence without significant overfitting, indicating effective generalization.

Test Performance:

LSTM Model Performance

The LSTM model achieved an accuracy of **78.01%** and a macro-averaged F1-score of **0.60** on the test set. It performed robustly in identifying downward market movements (Class 0), with a precision and recall of **0.87**, leading to an F1-score of **0.87**. However, performance declined for upward price trends (Class 1), where the model achieved a precision of **0.32**, recall of **0.33**, and an F1-score of **0.33**. The reduced effectiveness in this class is likely due to class imbalance despite the use of class weighting.

Summary:

Accuracy: **78.01%**

Macro-Averaged F1-Score: **0.60**

Class 0 (Downtrend):

Precision: **0.87**

Recall: **0.87**

F1-Score: **0.87**

Class 1 (Uptrend):

Precision: **0.32**

Recall: **0.33**

F1-Score: **0.33**

CNN Model Performance

The CNN model attained a test accuracy of **54.99%** and a macro-averaged F1-score of **0.49**. It showed high precision (**0.89**) when predicting downward trends (Class 0), but with a relatively lower recall of **0.53**, resulting in an F1-score of **0.66**. For upward price movements (Class 1), the model achieved a recall of **0.65** but a low precision of **0.21**, producing an F1-score of **0.32**. These results suggest the CNN model had difficulty generalizing and was more prone to false positives for upward trend predictions.

Summary:

Accuracy: **54.99%**

Macro-Averaged F1-Score: **0.49**

Class 0 (Downtrend):

Precision: **0.89**

Recall: **0.53**

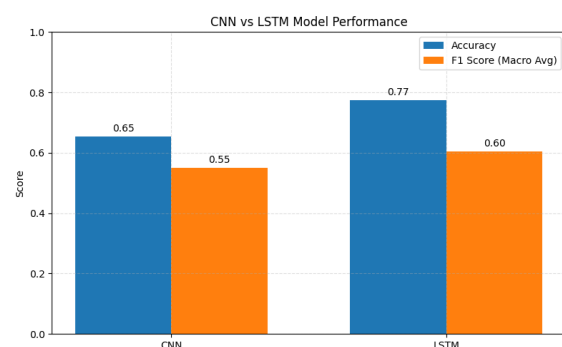
F1-Score: **0.66**

Class 1 (Uptrend):

Precision: **0.21**

Recall: **0.65**

F1-Score: **0.32**



6. Conclusion and Future directions

This study compared two deep learning architectures—an enhanced residual CNN and a dual-layer LSTM—for predicting directional movements in the S&P 500 index using historical stock price data.

The **LSTM model significantly outperformed the CNN**, achieving an accuracy of **78.01%** and a **macro-averaged F1-score of 0.60**. Its ability to capture long-term temporal dependencies made it particularly effective at identifying downward market trends.

In contrast, the **CNN model achieved 54.99% accuracy** and a **macro-averaged F1-score of 0.49**. While it demonstrated high precision for downward movements, it suffered from poor generalization and struggled with upward trend predictions due to class imbalance.

Notably, the CNN model showed better recall for upward trends (Class 1) but at the cost of a high false positive rate, misclassifying a substantial number of downward trends (Class 0), which undermines its reliability in real-world settings. The performance gap underscores the LSTM model's strength in sequence modeling, making it more suitable for financial time-series data. However, both models exhibited underperformance on the minority class (Class 1), highlighting the need for further optimization.

Recommended Enhancements:

Apply class balancing techniques such as SMOTE, oversampling, or focal loss to address data imbalance.

Incorporate external contextual features, including sentiment analysis from financial news or macroeconomic indicators.

Explore advanced architectures, such as hybrid CNN-LSTM models or attention-based transformers, to improve model interpretability and directional prediction accuracy.

In conclusion, while the LSTM model provides a strong foundation for forecasting downward trends, future iterations must enhance sensitivity to upward movements to create a more balanced and actionable prediction system for use in financial decision-making.

7. References

- [1]<https://www.sciencedirect.com/science/article/abs/pii/S0957417419301915>
- [2][Predicting stock market index using LSTM -ScienceDirect](#)
- [3][CNNpred: CNN-based stock market prediction using a diverse set of variables - UCI Machine Learning Repository](#)